

MI AEC API

Version 1.1

© 2020 SigmaStar Technology Corp. All rights reserved.

SigmaStar Technology makes no representations or warranties including, for example but not limited to, warranties of merchantability, fitness for a particular purpose, non-infringement of any intellectual property right or the accuracy or completeness of this document, and reserves the right to make changes without further notice to any products herein to improve reliability, function or design. No responsibility is assumed by SigmaStar Technology arising out of the application or use of any product or circuit described herein; neither does it convey any license under its patent rights, nor the rights of others.

SigmaStar is a trademark of SigmaStar Technology Corp. Other trademarks or names herein are only for identification purposes only and owned by their respective owners.

REVISION HISTORY

Revision No.	Description	Date
1.0	● Initial release	09/08/2020
1.1	● Modified API	10/15/2020

TABLE OF CONTENTS

REVISION HISTORY	i
TABLE OF CONTENTS	ii
1. OVERVIEW	1
1.1. Module Description	1
1.2. Keyword	1
1.3. Note	2
2. API REFERENCE	3
2.1. API List	3
2.2. IaaAec_GetBufferSize	3
2.3. IaaAec_Init	4
2.4. IaaAec_Config	5
2.5. IaaAec_Reset	5
2.6. IaaAec_Free	6
2.7. IaaAec_Run	7
2.8. IaaAec_GetLibVersion	11
3. AEC DATA TYPE	12
3.1. AEC data type list	12
3.2. AudioAecInit	12
3.3. AudioAecConfig	13
3.4. IAA_AEC_SAMPLE_RATE	15
3.5. IAA_AEC_BOOL	15
3.6. AEC_HANDLE	16
4. ERROR CODE	17

1. OVERVIEW

1.1. Module Description

AEC (short for Acoustic Echo Cancellation) is a function used to suppress far-end echo.

Echo is common in conference systems, building intercom, security monitoring and other scenarios. When the far-end sound is played from the speaker, the microphone re-collects the sound played by the speaker and transmits it back to the far-end after a very small delay, which will cause an echo to be heard at the far-end.

1.2. Keyword

- Far-end signal (Sin)

The signal source of the device speaker, or the signal from the remote.

- Near-end signal (Rin)

The signal recorded by the microphone of the device, which may include acoustic echo and near-end talker signals.

- Acoustic echo (AE)

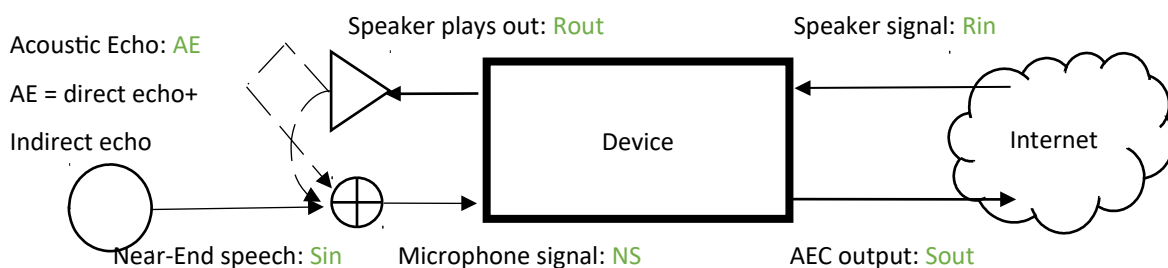
The sound played from the speaker of the device is transmitted back to the microphone through a direct or indirect path, and the echo is generated. The far-end talker will hear his echo after a certain delay after speaking. This echo is claimed as an acoustic echo (AE).

- Single talk

Only the speaker on the device side plays the far-end signal ($S_{in} = AE + NS$ when $NS = 0$)

- Double talk

While the speaker on the device side is playing, the near-end talker also speaks into the microphone ($S_{in} = AE + NS$ when $NS \neq 0$)



1.3. Note

In order to facilitate debugging and confirm the effect of the algorithm, the user application needs to implement replacement algorithm parameter and grab audio data.

2. API REFERENCE

2.1. API List

API name	Features
IaaAec_GetBufferSize	Get the memory size required for Aec algorithm running
IaaAec_Init	Initialize the memory required for the Aec algorithm
IaaAec_Config	Configure Aec algorithm
IaaAec_Reset	Reinitialize the memory of Aec algorithm
IaaAec_Free	Release Aec algorithm resources
IaaAec_Run	Aec algorithm processing
IaaAec_GetLibVersion	Get the version information of the Aec algorithm

2.2. IaaAec_GetBufferSize

➤ Features

Get the memory size required for Aec algorithm running.

➤ Syntax

```
unsigned int IaaAec_GetBufferSize(void);
```

➤ Parameters

Parameter Name	Description	Input/Output
----------------	-------------	--------------

➤ Return value

Return value is the memory size required for Aec algorithm running.

➤ Dependency

- Header: AudioAecProcess.h
- Library: libAEC_LINUX.so/ libAEC_LINUX.a

※ Note

The interface only returns the required memory size, and the application and release of memory need to be processed by the application.

➤ Example

None

2.3. IaaAec_Init

➤ Features

Initialize the memory required for the Aec algorithm.

➤ Syntax

```
AEC\_HANDLE IaaAec_Init (char* working_buffer_address, AudioAecInit * aec_init);
```

➤ Parameters

Parameter Name	Description	Input/Output
working_buffer_address	Memory address used by Aec algorithm	input
aec_init	Aec algorithm initialization structure pointer	input

➤ Return value

Return value	Result
0	Successful
Non-zero	Failed, refer to error code

➤ Dependency

- Header: AudioAecProcess.h
- Library: libAEC_LINUX.so / libAEC_LINUX.a

※ Note

- Aec algorithm only supports 8K/16K sampling rate.

➤ Example

None

2.4. IaaAec_Config

➤ Features

Configure Aec algorithm.

➤ Syntax

```
ALGO_AEC_RET IaaAec_Config(AEC\_HANDLE handle, AudioAecConfig *aec_config);
```

➤ Parameters

Parameter Name	Description	Input/Output
handle	Aec algorithm handle	input
aec_config	Aec algorithm configuration parameter structure pointer	input

➤ Return value

Return value	Result
0	Successful
Non-zero	Failed, refer to error code

➤ Dependency

- Header: AudioAecProcess.h
- Library: libAEC_LINUX.so / libAEC_LINUX.a

※ Note

None

➤ Example

None

2.5. IaaAec_Reset

➤ Features

Reinitialize the memory of Aec algorithm.

➤ Syntax

[AEC_HANDLE](#) IaaAec_Reset(char* working_buffer_address, [AudioAecInit](#) * aec_init);

➤ Parameters

Parameter Name	Description	Input/Output
working_buffer_address	Memory address used by Aec algorithm	input
aec_init	Aec algorithm initialization structure pointer	input

➤ Return value

Return value	Result
0	Successful
Non-zero	Failed, refer to error code

➤ Dependency

- Header: AudioAecProcess.h
- Library: libAEC_LINUX.so/ libAEC_LINUX.a

※ Note

- Reinitialize the Aec algorithm for the same algorithm handle (same memory address) (that is, reset the sampling rate and the number of near-end and far-end channels)
- After reinitializing the Aec algorithm, it needs to be reconfigured

➤ Example

None

2.6. IaaAec_Free

➤ Features

Release Aec algorithm resources

➤ Syntax

ALGO_AEC_RET IaaAec_Free([AEC_HANDLE](#) handle);

➤ Parameters

Parameter Name	Description	Input/Output
handle	Aec algorithm handle	input

➤ Return value

Return value	Result
0	Successful
Non-zero	Failed, refer to error code

➤ Dependency

- Header: AudioAecProcess.h
- Library: libAEC_LINUX.so/ libAEC_LINUX.a

※ Note

Must call IaaAec_Free first, and then release the memory used by the Aec algorithm.

➤ Example

None

2.7. IaaAec_Run

➤ Features

Aec algorithm processing

➤ Syntax

```
ALGO_AEC_RET IaaAec_Run(AEC\_HANDLE handle, short* pss_audio_near_end, short* pss_audio_far_end);
```

➤ Parameters

Parameter Name	Description	Input/Output
handle	Algorithm handle	input
pss_audio_near_end	Near-end data pointer	input/output
pss_audio_far_end	Far-end data pointer	input

➤ Return value

Return value	Result
0	Successful
Non-zero	Failed, refer to error code

➤ Dependency

- Header: AudioAecProcess.h
- Library: libAEC_LINUX.so/ libAEC_LINUX.a

※ Note

- The amount of near-end and far-end data must correspond to the point_number (the number of sampling points required for one IaaAec_Run) when calling IaaAec_Init.
- The result after AEC is output by pss_audio_near_end

➤ Example

```

1. #include <stdio.h>
2. #include <string.h>
3. #include <sys/time.h>
4. #include <stdlib.h>
5.
6. #include "AudioAecProcess.h"
7.
8. /* 0:Fixed input file 1:User input file */
9. #define IN_PARAMETER 1
10.
11. float AVERAGE_RUN(int a)
12. {
13.     static unsigned int num = 0;
14.     static float avg = 0;
15.     if(0 == num)
16.         avg = 0;
17.     num++;
18.     avg = avg + ((float)a - avg) / ((float)num);
19.     return avg;
20. }
21.
22. unsigned int _OsCounterGetMs(void)
23. {
24.     struct timeval t1;
25.     gettimeofday(&t1, NULL);
26.     unsigned int T = ((1000000 * t1.tv_sec) + t1.tv_usec) / 1000;
27.     return T;
28. }
29.
30. int main(int argc, char *argv[])
31. {
32.     short in_output[1024];
33.     short input_far[1024];
34.     char src_file1[128] = {0};
35.     char src_file2[128] = {0};
36.     char dst_file[128] = {0};
37.     unsigned int workingBufferSize;
38.     char *workingBuffer = NULL;
39.     int counter = 0;
40.     ALGO_AEC_RET ret;
41.     int tempSize;
42.     unsigned int T0, T1;
43.     float avg;
44.     FILE *fpInFar, *fpOut, *fpInNear;
45.     AudioAecInit aec_init;
46.     AudioAecConfig aec_config;
47.     AEC_HANDLE handle;
48.     /*****User change section start*****/
49.     unsigned int supMode_band[6] = {20,40,60,80,100,120};
50.     unsigned int supMode[7] = {4,4,4,4,4,4,4};
51.     aec_init.point_number = 128;
52.     aec_init.nearend_channel = 1;
53.     aec_init.farend_channel = 1;
54.     aec_init.sample_rate = IAA_AEC_SAMPLE_RATE_16000;
55.     aec_config.delay_sample = 0;
56.     aec_config.comfort_noise_enable = IAA_AEC_FALSE;
57.     /*****User change section end*****/
58.     memcpy(&(aec_config.suppression_mode_freq[0]), supMode_band, sizeof(supMode_band));
59.     memcpy(&(aec_config.suppression_mode_intensity[0]), supMode, sizeof(supMode));

```

```
60.  //(1)IaaAec_GetBufferSize
61.  workingBufferSize = IaaAec_GetBufferSize();
62.  workingBuffer = (char*)malloc(workingBufferSize);
63.  if(NULL == workingBuffer)
64.  {
65.      printf("workingBuffer malloc failed !\n");
66.      return -1;
67.  }
68.  printf("workingBuffer malloc success !\n");
69.  //(2)IaaAec_Init
70.  handle = IaaAec_Init(workingBuffer, &aec_init);
71.  if (NULL == handle)
72.  {
73.      printf("AEC init failed !\r\n");
74.      return -1;
75.  }
76.  printf("AEC init success !\r\n");
77.  //(3)IaaAec_Config
78.  ret = IaaAec_Config(handle, &aec_config);
79.  if(ret)
80.  {
81.      printf("IaaAec_Config failed !\n");
82.      return -1;
83.  }
84.  printf("IaaAec_Config succeed !\n");
85.
86. #if IN_PARAMETER
87.  if(argc < 4)
88.  {
89.      printf("Please enter the correct parameters!\n");
90.      return -1;
91.  }
92.  sscanf(argv[1], "%s", src_file1);
93.  sscanf(argv[2], "%s", src_file2);
94.  sscanf(argv[3], "%s", dst_file);
95. #else
96.  sprintf(src_file1, "%s", "./farend.wav");
97.  sprintf(src_file2, "%s", "./nearend.wav");
98.  if(argc < 2)
99.  {
100.      printf("Please enter the correct parameters!\n");
101.      return -1;
102.  }
103.  sscanf(argv[1], "%s", dst_file);
104. #endif
105.
106.  fpInFar = fopen(src_file1, "rb");
107.  if(NULL == fpInFar)
108.  {
109.      printf("src_file1 open failed !\n");
110.      return -1;
111.  }
112.  printf("src_file1 open succeed !\n");
113.  fpInNear = fopen(src_file2, "rb");
114.  if(NULL == fpInNear)
115.  {
116.      printf("src_file2 open failed !\n");
117.      return -1;
118.  }
119.  printf("src_file2 open succeed !\n");
120.  fpOut = fopen(dst_file, "wb");
121.  if(NULL == fpOut)
```

```

122.     {
123.         printf("dst_file open failed !\n");
124.         return -1;
125.     }
126.     printf("dst_file open succeed !\n");
127.     #if 1
128.         fread(in_output, sizeof(char), 44, fpInNear); // Remove the 44 bytes header
129.         fwrite(in_output, sizeof(char), 44, fpOut); // New file add header
130.         fread(input_far, sizeof(char), 44, fpInFar); // Remove the 44 bytes header
131.     #endif
132.     tempSize = aec_init.point_number * aec_init.nearend_channel;
133.     while(fread(in_output, sizeof(short), tempSize, fpInNear))
134.     {
135.         tempSize = aec_init.point_number * aec_init.farend_channel;
136.         fread(input_far, sizeof(short), tempSize, fpInFar);
137.         counter++;
138.         T0 = (long)_OsCounterGetMs();
139.         //(4)IaaAec_Run
140.         ret = IaaAec_Run(handle, in_output, input_far);
141.         T1 = (long)_OsCounterGetMs();
142.         avg = AVERAGE_RUN(T1 - T0);
143.         if(0 == counter % 100)
144.         {
145.             printf("counter = %d\n", counter);
146.             printf("current time = %f\n", (float)counter * aec_init.point_number / aec_ini
t.sample_rate);
147.             printf("process time = %lu(ms)\t", (long)(T1 - T0));
148.             printf("AVG is %.2f ms\n", avg);
149.         }
150.         if(ret < 0)
151.         {
152.             printf("IaaAec_Run failed !\n");
153.             break;
154.         }
155.         tempSize = aec_init.point_number * aec_init.nearend_channel;
156.         fwrite(in_output, sizeof(short), tempSize, fpOut);
157.     }
158.     //(5)IaaAec_Free
159.     IaaAec_Free(handle);
160.     fclose(fpInFar);
161.     fclose(fpInNear);
162.     fclose(fpOut);
163.     free(workingBuffer);
164.     printf("AEC end !\n");
165.
166.     return 0;
167. }

```

2.8. IaaAec_GetLibVersion

➤ Features

Get the version information of the Aec algorithm.

➤ Syntax

```
ALGO_AEC_RET IaaAec_GetLibVersion(unsigned short *ver_year,
                                   unsigned short *ver_date,
                                   unsigned short *ver_time);
```

➤ Parameters

Parameter Name	Description	Input/Output
ver_year	Year when Aec Algorithm Library was built	output
ver_date	Date when Aec Algorithm Library was built	output
ver_time	Date when Aec Algorithm Library was built	output

➤ Return value

Return value	Result
0	Successful
Non-zero	Failed, refer to error code

➤ Dependency

- Header: AudioAecProcess.h
- Library: libAEC_LINUX.so/ libAEC_LINUX.a

➤ Example

None

3. AEC DATA TYPE

3.1. AEC data type list

The AEC module related data types are defined as follows:

DATA TYPE	Description
AudioAecInit	The initialization parameter structure of the Aec algorithm
AudioAecConfig	the configuration parameter structure of the Aec algorithm
IAA_AEC_SAMPLE_RATE	AEC algorithm supported sampling rate type
IAA_AEC_BOOL	The Boolean type used internally in the Aec algorithm
AEC_HANDLE	The handle type of the Aec algorithm

3.2. AudioAecInit

➤ Description

Define the initialization parameter structure of the Aec algorithm.

➤ Definition

```
typedef struct
{
    unsigned int point_number;
    unsigned int nearend_channel;
    unsigned int farend_channel;
    IAA_AEC_SAMPLE_RATE sample_rate;
}AudioAecInit;
```

➤ Member

Member name	Description
point_number	The number of sampling points processed by the Aec algorithm once
nearend_channel	The far-end channel number (the microphone channel number)
farend_channel	The near-end channel number (the speaker channel number)
sample_rate	Sampling rate processed by Aec algorithm

※ Note

None

➤ Related data types and interfaces

[IaaAec_Init](#)

[IaaAec_Reset](#)

3.3. AudioAecConfig

➤ Description

Define the configuration parameter structure of the Aec algorithm

➤ Definition

```
typedef struct
{
    IAA_AEC_BOOL comfort_noise_enable;
```

```

short delay_sample;
unsigned int suppression_mode_freq[6];
unsigned int suppression_mode_intensity[7];
}AudioAecConfig;

```

➤ Member

Member name	Description
comfort_noise_enable	Add comfort noise after AEC to avoid the muting part of the paragraph after echo cancellation, which makes users feel disconnected. If noise reduction is turned on after AEC, it is recommended to turn on comfort noise to help NR converge noise.
delay_sample	Echo delay samples between left and right channels When Stereo cost down is turned on, you can set the sample delay number according to the relative position of the microphone, reducing the amount of calculation required for the dual-channel. Default value : 0 Unit : the number of samples Value ranges: -9-9
suppression_mode_freq	The frequency band division of AEC processing divides the highest frequency that can be resolved by different sampling rates into 128 equal parts. Setting six values, which can cut out seven segments to set different AEC intensities. Value ranges: 1-127
suppression_mode_intensity	Set seven intensities respectively in the seven paragraphs cut by the suppression_mode_freq. Value ranges: 0-25

※ Note

- comfort_noise_enable

After enabling the parameter, the AEC algorithm will add some comfort noise when there is no sound. This function can be enabled when you require a stable noise floor or when AEC is too clean that causes the NR convergence time is too long.

- delay_sample

Due to the placement of the microphone and the speaker, the distance between the microphones, etc., the time points when the left and right channels receive the echo are not consistent, and the echo delay between the two channels is different.

This value indicates how many sampling points the left channel receives the echo earlier than the

right channel. You can adjust this value to align the left and right channel data. When setting, please use the left channel as a reference. For example, the echo of the left channel is 4 samples faster than the right, `s16DelaySample` is set to 4, otherwise it is set to -4.

- `suppression_mode_freq`

This array divides the highest frequency of the current sampling rate into 7 frequency bands for processing. The following is the conversion formula of array elements and frequency range: f_i . The array requires that each element must be greater than the previous element.

- `suppression_mode_intensity`

This array represents the echo cancellation intensity of each frequency band, and each element corresponds to the 7 frequency bands divided by `u32AecSupfreq`. The greater the intensity, the more details are eliminated and the more unnatural the sound.

➤ Related data types and interfaces

[IaaAec_Config](#)

3.4. IAA_AEC_SAMPLE_RATE

➤ Description

AEC algorithm supported sampling rate type.

➤ Definition

```
typedef enum
{
    IAA_AEC_SAMPLE_RATE_8000 = 8000 ,
    IAA_AEC_SAMPLE_RATE_16000 = 16000 ,
}IAA_AEC_SAMPLE_RATE;
```

➤ Member

Member name	Description
IAA_AEC_SAMPLE_RATE_8000	8K sampling rate
IAA_AEC_SAMPLE_RATE_16000	16K sampling rate

※ Note

None

- Related data types and interfaces

[AudioAecInit](#)

3.5. IAA_AEC_BOOL

- Description

Define the Boolean type used internally in the Aec algorithm.

- Definition

```
typedef enum
{
    IAA_AEC_TRUE = 1,
    IAA_AEC_FALSE = 0,
}IAA_AEC_BOOL;
```

- Member

Member name	Description
IAA_AEC_TRUE	True
IAA_AEC_FALSE	False

- ※ Note

None

- Related data types and interfaces

[AudioAecConfig](#)

3.6. AEC_HANDLE

- Description

Define the handle type of the Aec algorithm.

- Definition

```
typedef void* AEC_HANDLE;
```

➤ Member

Member name	Description
-------------	-------------

※ Note

None

➤ Related data types and interfaces

[IaaAec_Init](#)[IaaAec_Config](#)[IaaAec_Reset](#)[IaaAec_Free](#)[IaaAec_Run](#)

4. ERROR CODE

AEC API error codes are shown as follow:

Error code	Definition	Description
0x00000000	ALGO_AEC_RET_SUCCESS	AEC runs successfully
0x10000401	ALGO_AEC_RET_INIT_ERROR	AEC isn't initialized
0x10000402	ALGO_AEC_RET_INVALID_HANDLE	HANDLE is invalid
0x10000403	ALGO_AEC_RET_INVALID_SAMPLE_RATE	Sampling frequency doesn't support
0x10000404	ALGO_AEC_RET_INVALID_POINT_NUMBER	Points per frame doesn't support
0x10000405	ALGO_AEC_RET_INVALID_CHANNEL	Channel number doesn't support
0x10000406	ALGO_AEC_RET_INVALID_SUP_BAND	AEC rejection band parameter setting is invalid
0x10000407	ALGO_AEC_RET_INVALID_SUP_MODE	AEC rejection intensity parameter setting is invalid
0x10000408	ALGO_AEC_RET_INVALID_COMFORT_NOISE	AEC comfort noise parameter setting is invalid
0x10000409	ALGO_AEC_RET_INVALID_DELAY_SAMPLE	AEC delay sample number setting is invalid
0x10000410	ALGO_AEC_RET_API_CONFLICT	Other APIs are running
0x10000411	ALGO_AEC_RET_INVALID_CALLING	Incorrect order of calling API